

김재형 · AI Application Developer / Backend Engineer

실패의 원인을 깊이 파고들어 해결하는 것을 좋아하는 개발자

AI Application Developer · Backend Engineer

kkuldangi2@gmail.com GitHub ↗ Blog ↗ LinkedIn ↗

SUMMARY

RAG 근거 선택: 근거 충족률 66.25% → 91.25%, 잘못된 근거 20 → 0건

에이전트 실행 환경: 사용자 정의 Skill과 승인 기반 실행, 편집 품질 94/114 → 104/114

데이터 파이프라인: PDF 변환 통과율 45.17% → 89.89%, 약품 변환 비용 예상 \$200 대비 \$11.18

PROJECTS & EXPERIENCE

Fruition

2026.04 — 현재

LLM Wiki 기반 AI 워크스페이스 · AI SW 마에스트로 17기 · AWS EKS 배포 완료, QA 진행 중

팀 프로젝트 (3명 + 디자이너) / AI 파트 리드

시작 계기: 팀원이 제기한 문서 관리의 불편함을 디자인 싱킹으로 구체화. 저장만 해 둔 문서를 필요할 때 찾지 못하는 문제를 풀고자 했습니다.

팀 구성: 본인(AI 전체), 백엔드 1명(Spring 백엔드 전체), 프론트엔드·DevOps 1명(프론트엔드, MSA 기반 AWS 배포), 외주 디자이너 · 멘토 3명

GitHub · AI 분리 저장소 ↗

RAG 과정의 검색 정확도 개선

저장된 문서에서 AI 답변에 필요한 근거를 찾는 기능입니다. 답이 있는 80문항의 근거 충족률을 66.25% → 91.25%로 높이고, 답 없는 20문항의 잘못된 근거 반환을 20 → 0건으로 줄였습니다.

- 답변 근거 선택 개선

문제: 기존 검색이 필요한 근거를 놓치고, 답이 없는 질문에도 근거를 반환.

해결: 라우팅에서 효과를 본 선택형 판단 모델 Jev를 RAG에 적용. 전체 근거 실행 후 동일 후보·반향량으로 비교 조건을 맞추고, 후보 300개·최대 4병렬로 조정.

성과: 논문 9편의 평가에서 답이 있는 80문항의 근거 충족률 66.25% → 91.25%. 답 없는 20문항의 잘못된 근거 반환 20 → 0건. Jev 최적화 전후 시간 중앙값 9.090초 → 2.251초.

Skill 기능 구현과 AI Agent 라우팅 성능 개선

반복 작업을 Skill로 저장·재사용하고 실제 변경은 사용자 승인 후 반영하도록 구현했습니다. 요청을 질문·편집 등으로 분류하는 라우팅에는 Jev를 적용해 정답 77/98 → 81/98건, 판단시간 중앙값 8.967초 → 0.658초로 개선했습니다.

- 사용자 정의 작업 지침(Skill)

문제: 사용자마다 다른 반복 작업을 개발자가 모두 기능으로 추가하기 어려움.

해결: 자연어 지침의 작성·검토·게시 기능과 권한·승인 우회 검사를 구현. 실제 문서 변경은 미리보기를 확인한 뒤 별도로 승인하도록 설계.

성과: 게시한 Skill을 Agent에서 사용할 수 있도록 구현. 문서 6개 검증에서 승인 전 변경 없이, 승인 후 폴더 3개 생성과 문서 이동 확인.

- Jev 기반 요청 분류 개선

문제: 사용자 질문을 문서 편집으로 분류하는 등 잘못된 작업으로 연결.

해결: 질문·편집·문서 생성·폴더 정리 등 작업 종류와 부가 조건을 고르는 라우팅에 선택형 판단 모델 Jev(정해진 선택지 중에서만 답을 고르는 모델) 적용.

성과: 동일 요청 98개에서 작업 종류와 부가 조건을 모두 맞춘 수 77 → 81건(같은 선택지를 준 GPT-5-nano 63건). 판단시간 중앙값 8.967초 → 0.658초.

대화 맥락과 편집 입력 구성

대화가 길어져도 맥락을 유지하고, 문서 편집에서 모델에 보내는 입력을 줄이는 작업입니다. 최근 여섯 턴은 원문으로 두고 그 이전은 누적 요약으로 압축했습니다.

- 대화 메모리

문제: 대화가 길어지면 이전 턴을 전부 모델에 넣을 수 없음.

해결: 최근 여섯 턴은 원문 유지, 그 이전은 누적 요약으로 압축. 요약 호출 실패 시 이전 요약을 유지. 라우터에는 요약·최근 메시지와 이전 턴의 라우팅 결정까지 전달.

성과: 지시어가 가리키는 대상을 이전 턴에서 찾아 라우팅에 반영.

- 편집 입력 범위 제한

문제: 문서 전체를 모델에 보내면 비용과 지연이 커짐.

해결: 편집 대상 줄 범위와 위아래 문맥만 전달하고, 표·코드 블록 경계를 걸치는 범위는 거절. 원문 대비 입력 비율과 처리 시간 평균·p95를 벤치마크로 확인.

성과: 문서 길이와 무관하게 편집 입력을 대상 범위 중심으로 유지.

문서 확장자 변환 파이프라인 구축과 Wiki 생성 시간 개선

사용자가 올린 PDF를 검색과 답변에 활용하도록 변환하고, 여러 문서의 Wiki 생성 시간을 줄였습니다. PDF 30쪽의 내부 평가 통과율은 45.17% → 89.89%, 문서 4개 분석 시간은 282.11

초 → 73.89초로 개선했습니다.

• PDF → Markdown 변환

문제: 오픈소스 변환에서 표·수식이 손상됐고, 전체 AI 변환에도 오류와 호출 부담이 남음.

해결: PDF 변환 오픈소스를 비교해 AnyDoc 채택. 깨진 본문만 AI로 복원. 표·수식은 별도 처리하고 그림은 원본 보존.

성과: PDF 30쪽의 내부 평가 통과율 45.17% → 89.89%.

• 문서의 Wiki 생성 병렬화

문제: 앞선 문서의 외부 AI 응답을 기다리는 동안 뒤의 문서 처리가 지연.

해결: Worker 수와 내부 동시 요청 수를 각각 변경하며 비교. 메모리 증가를 고려해 Worker 최대 4개를 채택하고, 오히려 느려진 추가 동시 요청, GPU 병렬화는 제외.

성과: 문서 4개 분석 시간 282.11초 → 73.89초(약 74% 감소)

서비스별 책임과 통신 구조, LLM 추적 환경 구성

AI 기능을 포함한 서비스의 책임과 데이터 소유권, 서비스 간 통신 방식을 정하고, AI 실행 과정을 단계 단위로 확인할 수 있는 추적 환경을 붙이는 작업입니다.

• MSA 초기 설계 제한 후 팀원·멘토와 데이터 소유권, 서비스 경계, 통신 방식을 논의해 최종 구조 공동 설계.

• LLM 실행 추적과 민감정보 경계

문제: 평가 루프가 일반 Python 흐름이라 의미 추출·평가·재시도 단계를 나눠 볼 수 없었고, 추적 도구에 보내는 내용에 민감정보가 섞일 수 있었음.

해결: 평가 루프를 LangGraph 그래프로 재구성해 응답 생성·평가·재시도를 노드 단위로 추적. LangSmith 키가 없으면 그래프 실행은 유지하고 추적만 생략. Agent 그래프는 추적을 끄고 도구 조회 원문을 상태 저장에서 제외하며, 보관 기간이 끝나면 저장 상태를 먼저 삭제. 자체 프롬프트 로그는 기록 전에 숫자 개인정보를 마스킹.

성과: 요청 실행에서 그래프 노드와 모델 호출 구간을 확인. 시간을 재는 실험에서는 추적과 프롬프트 로그를 모두 끄고 실행해 계속 부담이 측정값에 섞이지 않게 했습니다.

작업 취소와 저장한 변경의 복구

AI가 이미 저장한 변경을 되돌리고, 무엇을 되돌렸는지 남기는 작업입니다. 여러 저장소에 걸친 실행을 하나의 트랜잭션으로 묶을 수 없어 역작업으로 복구했습니다.

• 취소한 작업의 역순 복구

문제: AI 작업이 모델 호출 사이에 문서·폴더·Wiki·Skill을 여러 번 저장해, 실행만 멈추면 일부 변경이 남음.

해결: 취소 상태를 먼저 저장해 후속 쓰기를 막고, 변경 전 snapshot과 승인에 연결된 맥등기로 자식부터 역순 복구. Agent 턴과 복구가 같은 실행 잠금을 쓰고, 본문은 새 revision으로 저장해 늦은 이벤트의 덮어쓰기를 방지. 복구는 모델 호출 없이 실행 기록만 사용.

성과: 전체 복구를 확인해야 취소가 확정되고, 충돌은 재시도 상태로 보존.

• 다른 작업의 기여를 지우지 않는 복구

문제: 개념 페이지가 여러 문서의 기여를 합친 결과라, 과거 snapshot으로 덮으면 다른 문서의 정상 기여까지 사라짐.

해결: 작업·페이지 revision·문서별 기여를 덧붙이기만 하는 이력으로 남기고, 복구 시 대상 기여만 제외해 재조립. 물리 삭제 대신 삭제 표시로 감사 이력 보존.

성과: 비동기 복구 중 상태 변화를 revision과 기여 hash로 검증하고, 복구 자체도 새 이력으로 기록.

수치는 내부 평가 기준이며, RAG 시간 단축은 Jev 최적화 전후 비교입니다. 문서 병렬 처리 시간은 저장·후처리를 제외한 분석 단계 기준입니다.

문제 해결 과정과 실험 결과는 상세 포트폴리오에서 볼 수 있습니다.

[RAG 근거 선택](#) [에이전트 실행 환경](#) [PDF 변환](#) [문서 병렬 처리](#)

Pilltip

2025.03 — 2025.12

개인 맞춤 AI 안심 복약 솔루션 · 부산대학교 졸업과제

팀 프로젝트 (3명 + 디자이너) / 핵심 백엔드 · 데이터 정제

시작 계기: 약품 정보가 흩어져 있어, 지금 건강 상태에서 특정 약을 먹어도 되는지 확인하기 어렵다는 문제에서 출발했습니다.

팀 구성: 본인(백엔드-AI), Android 1명(Kotlin 앱 전반), 백엔드·보안 1명(사용자 DB, 로그인, 문진표, AES-GCM), 디자이너. 아래 목록은 제가 구현한 기능입니다.

[GitHub 저장소](#)

데이터 정제 파이프라인 구축

약 4만 4천 건의 의약품 원문을 쉬운 설명으로 바꾸는 작업입니다. 중복 제거로 변환 대상을 1GB → 326MB로 줄이고 누락 정보를 보완한 뒤, 전체 원문 처리 예상 약 \$200 대비 실제 API 지출 \$11.18로 변환했습니다.

• **문제:** 약 4만 4천 건의 의약품 원문을 모두 쉬운 설명으로 변환하면 API 비용 약 \$200 예상.

해결: 완전 일치 문장 → 의미가 같은 문장 순으로 중복 제거. 누락된 5%는 성분별 주의사항으로 보완하고, GPT Batch API로 변환한 뒤 약품별로 재연결.

성과: 변환 대상 1GB → 326MB. 전체 원문 변환 예상 비용 약 \$200 대비 실제 API 지출 \$11.18.

비용은 예상치와 실제출의 비교이며 전처리 인건비는 제외했습니다.

개인 건강 정보 기반 답변 제공 AI 복약 챗봇 개발

사용자가 증상을 자연어로 물으면 관련 약품과 복약 정보를 찾는 기능입니다. 건강 프로파일은 내부에서 판단해 개인화에 활용하고, 외부 AI에 보내는 정보와 분리했습니다.

• **문제:** 일상적인 증상 표현과 약품 효능의 용어가 다르고, 개인화에 필요한 건강 정보는 외부 AI에 전달하기 어려움.

해결: Weaviate 의미 검색과 AI 오케스트레이션으로 약품 탐색·복약 위험·섭취량별 처리 경로 구성. 임신 여부·복용약·기저질환은 내부 코드로 판단.

성과: 자연어 증상으로 약품 정보를 탐색하고, 해당 건강 프로필을 외부 LLM 입력에서 제외한 개인화 흐름 구현.

백엔드 기능 구현

앱에서 약품을 찾고 복약 위험정보를 확인하며, 복약 알림·기록과 가족 프로필을 사용할 수 있도록 서버 기능을 구현했습니다. 모바일 앱과 사용자·보안 기능을 맡은 팀원과 연동했습니다.

• Java-Spring Boot 기반 의약품 DB 설계, 검색·자동완성과 복약 위험정보(DUR) 표출 구현.

• FCM 복약 알림·로그, 딥링크 친구 초대, 가족 프로필 전환을 구현하고, Notion에 API 문서·회의록·ADR을 남겨 의사결정을 공유했습니다.

전시 현장 피드백을 반영한 기능 확장

부산대학교 대표 전시팀으로 부스를 운영하며 받은 피드백을 다음 개발 범위에 반영했습니다. 첫 전시에서 나온 건강기능식품 요청을 보유한 데이터 범위에서 기능으로 연결했습니다.

- 건강기능식품 지원 확대
피드백: 첫 전시에서 의약품만이 아니라 건강기능식품도 함께 확인하고 싶다는 요청을 받았습니다.
반영: 보유한 건강기능식품 데이터 범위에서 검색·자동완성 대상에 추가하고, 복용 주의·병용 정보 표출, 복약 알람·기록 등록, 챗봇 답변 대상까지 확장했습니다.
범위: 확보한 데이터에 한정되며, 의약품과 같은 수준의 전수 정보는 아닙니다.
- 반영하지 못한 지적
실행 환경을 컨테이너로 옮기는 편이 낫다는 지적을 받았지만 졸업과제 기간 내에는 적용하지 못했습니다. 이후 Fruition에서는 컨테이너 기반 배포를 전제로 AI 서비스를 나눴습니다.

문제 해결 과정과 실험 결과는 상세 포트폴리오에서 볼 수 있습니다.

[데이터 파이프라인 사례](#) ↗ [개인화 설계 사례](#) ↗

기술

AI & Search LangChain · LangGraph · Spring AI · RAG · LLM Evaluation · Prompt Injection 대응
LangSmith · Elasticsearch · Weaviate

Backend Java · Spring Boot · Python

Data & Infra MySQL · PostgreSQL · Redis · Kafka
Docker · Docker Compose · GitHub Actions

활동 · 기여

- 2026.06 — 2026.06 **Rhwp 오픈소스 기여**
Rust 기반 HWP/HWPX 프로젝트의 오류 분석, 수정안 제출과 CI 대응. 그림·표·도형의 textFlow 속성이 저장 후 초기화되는 오류를 수정했습니다.
[GitHub 저장소](#) ↗ [PR #1213](#) · [병합](#) ↗ [PR #1351](#) · [제출](#) ↗
- 2025.03 — 2026.01 **부산대학교 APPTIVE**
Backend 멘티 및 멘토. 멘티 12명 대상 6회 멘토링과 코드 리뷰. HTTP·Servlet·REST API·DB 기초를 보강하는 커리큘럼 개편에 참여했습니다.
APPTIVE 백엔드 멘토 공로상 · 2026.01.09
- 2025.11 / 2025.07 **SK AI SUMMIT · K-ICT WEEK in Busan**
부산대학교 대표 전시팀으로 Pilltip 부스를 운영하고 서비스 시연과 기술 질의응답을 진행했습니다.
- 2022.02 — 2025.07 **백준 945일 연속 문제 해결**
하루 한 문제를 목표로 solved.ac 기준 최장 945일 연속 문제를 해결했습니다. 누적 1,659문제, solved.ac Platinum IV. 풀이는 GitHub 저장소에 기록했습니다.
[BaekJoon 풀이 저장소](#) ↗

학력 · 자격

- 2022.02 — 2026.02 (편입) **부산대학교**
평균 학점 4.12 / 4.5
- 2019.03 — 2022.02 (중퇴) **대구대학교**
평균 학점 4.2 / 4.5
- 2025.09 **정보처리기사**
한국산업인력공단

수상

- 2025.10 **캡스톤디자인 금상**
소프트웨어·인공지능 분야 / 부산대학교 정보의생명공학대학

2025.09



부산 DATA WEEK 최우수상
데이터 활용 우수사례 공모전 / 부산테크노파크

2025.09



AI LAUNCH 커리어스킬 창업톤 장려상
KRYPTON X x Root Impact x Google.org

2025.08



SW중심대학 디지털 경진대회 후원기업상
SW중심대학협의회